



UNIVERSIDADE
TÉCNICA DO
ATLÂNTICO



CAMPUS
DO MAR

INSTITUTO DE ENGENHARIAS E CIÊNCIAS DO MAR
Fundamentos de Sistemas Operativos
(LEIT)

Exercícios Práticos 1
Processos no Linux

30 de outubro de 2024

1. Crie um programa que, ao ser executado, use fork para criar um processo filho. Em seguida, exiba os IDs do processo pai (getppid) e do processo filho (getpid). O processo pai deve esperar pelo término do filho antes de encerrar.
2. Modifique o exercício anterior para que o processo pai crie três processos filhos. Cada filho deve exibir seu próprio ID e o ID do pai, enquanto o pai aguarda que todos os filhos terminem antes de encerrar.
3. Escreva um programa em que o processo pai cria um processo filho. O filho deve usar execl ou execp para executar outro programa qualquer, como /bin/date (para exibir a data e hora). O processo pai espera até que o filho termine e exibe uma mensagem após a execução.
4. Crie um programa onde o processo pai cria dois processos filhos. Cada filho deve esperar um tempo diferente (use sleep para simular o tempo). O pai deve esperar pelo filho que terminar primeiro usando wait e exibir o ID do filho que terminou.
5. Escreva um programa que crie um processo filho e faça o filho retornar um código de saída específico usando exit. O pai deve usar wait para capturar o código de saída do filho e exibir uma mensagem diferente dependendo do valor retornado.
6. Crie um programa onde o processo filho usa execp para tentar executar um comando inexistente, como invalid_command. Verifique como o programa lida com erros de execução, e o processo pai deve capturar e exibir a falha.
7. Crie um programa onde um processo pai cria um processo filho, e este processo filho cria outro processo filho. Exiba o ID de cada processo e seu pai, e garanta que o processo pai inicial só finalize após todos os processos terminarem.
8. EEscreva um programa que use fork para criar um processo filho, e, em seguida, o processo filho deve usar execp para executar um comando (como ps aux). O processo pai deve esperar o término do filho e exibir uma mensagem informando a conclusão do processo.

9. Crie um programa onde o processo pai e o processo filho se revezam exibindo mensagens de "ping" e "pong" um do outro (usando getpid para identificação). Use sleep para controlar a ordem, e o pai deve esperar pelo filho ao final.
10. Escreva um programa que cria um processo filho. O filho deve executar o comando date usando execl e, após a execução, o processo pai deve recriar o filho com outro fork, mas dessa vez o filho executa ls -la. O pai espera pelo término de ambos os comandos.
11. Crie um programa que receba um número N como entrada. O processo pai deve criar N processos filhos em sequência. Cada filho exibe seu PID e o PID do pai, e o pai espera por cada filho antes de terminar.
12. Crie um programa onde o processo pai cria dois processos filhos. O primeiro filho executa o comando ls, e o segundo executa pwd. O processo pai deve garantir que o segundo filho só inicie a execução após o término do primeiro.
13. Escreva um programa em que o processo filho usa execl para executar o comando echo, transmitindo uma lista de argumentos (como "Olá, mundo!"). O processo pai deve esperar pelo término do comando e exibir uma mensagem de confirmação.
14. Crie um programa que mede o tempo de execução de um processo filho. O pai cria um filho que dorme por alguns segundos (ex.: sleep 3) e depois exibe uma mensagem. O pai mede e exibe o tempo total que o filho levou para executar usando time.
15. Crie um programa onde o processo pai cria um filho que executa o comando date, que cria outro filho que executa whoami, e, por fim, cria um terceiro filho que executa hostname. Cada processo exibe o comando executado e seu PID antes de terminar.'